Full-Stack Take-Home: Snip

Oct 15, 2026 · @Abhishek Sharma

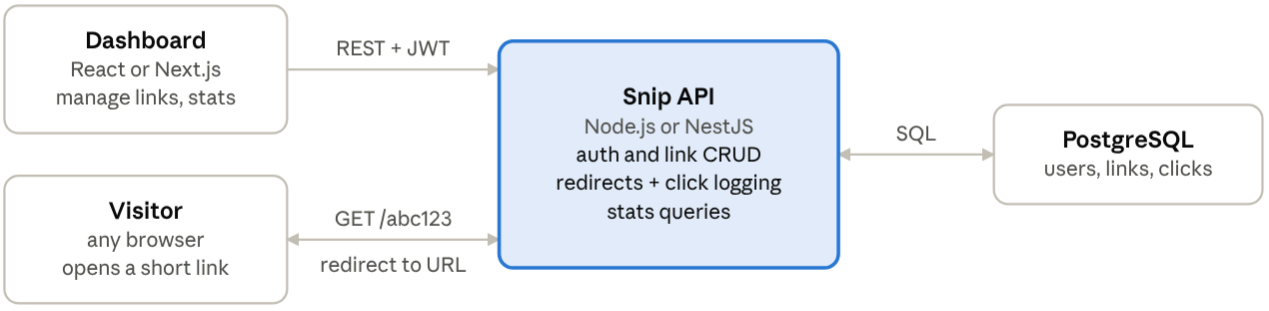
The task

Build **Snip**, a URL shortener with click stats, as two separately deployed apps: an API in Node.js or NestJS, and a dashboard in React or Next.js. Users sign in, shorten links, share them, and see how many times each one was opened.

It's deliberately small. We care more about how carefully the basics are done than how much you build.

- **Time:** 3 calendar days from when you receive this, due by 11:59 PM IST on day 3.
- **Effort:** about 6 to 8 focused hours.
- **Scope:** get the required parts working properly before touching any extras. If you run short, say in the README what's missing.
- **Questions:** if something's unclear, make a reasonable call, note it in the README, and keep going.

Visitors hit the API for redirects; the dashboard manages links through it



Snip architecture · dashboard, visitor, API, database

Short links live on the API's domain, so visitors never touch the frontend. The dashboard only talks to the API, never to the database.

Stack and ground rules

The frontend and backend must be two separate apps on two separate URLs. One repo with two folders or two repos, your call.

- **Backend:** Node.js (Express, Fastify or similar) or NestJS. TypeScript preferred.
- **Frontend:** React (Vite or similar) or Next.js. TypeScript preferred.

- **If you use Next.js**, it's the frontend only. No business logic in API routes, route handlers or server actions.
- **Database:** PostgreSQL preferred. MongoDB is fine if you explain why.
- **Write the auth yourself.** No Firebase, Supabase Auth, Auth0 or Clerk. Libraries like bcrypt, jsonwebtoken or Passport are fine.
- **AI coding tools are fine.** We'll ask you to explain and change your code live, so make sure you understand all of it.
- **Commit as you go.** Please don't squash everything into one commit.

Backend requirements

The API handles auth, link management, the redirects themselves and click stats. Paths below are suggestions; the behaviour is what we check.

Auth

- Sign up and log in with email and password. Hash passwords with bcrypt or argon2.
- JWT-based auth. Explain your expiry approach in the README.
- A user only ever sees and manages their own links. Another user's link ID returns 404.

Links

`POST /api/links`, `GET /api/links`, `PATCH /api/links/:id` and `DELETE /api/links/:id`.

Field	Rules
url	Required. A valid http or https URL, max 2048 characters
alias	Optional custom code. 4 to 30 characters: letters, numbers, hyphens. Unique, case-insensitive
expiresAt	Optional. Must be in the future

- No alias given: generate a random code of 6 to 8 characters. Codes must be unique, so handle the rare collision rather than hoping it never happens.
- Reject aliases that would clash with your own routes, like `api` or `health`.
- The list endpoint is paginated, newest first, and includes each link's total clicks.
- Editing can change the destination URL or expiry, not the code.

- Errors use the right status codes and one consistent JSON shape, e.g. 409 when an alias is taken.

Redirects

- `GET /:code` on the API's domain redirects to the destination, e.g. `https://your-api.example/abc123`.
- Unknown code returns 404. Expired or deleted link returns 410. A simple HTML page for these is nice; JSON is fine.
- Pick 301 or 302 and explain why in the README.
- Every redirect records a click with its time, referrer and user agent.
- Counts must stay accurate when many people open the same link at once. We'll test this.

Stats

- `GET /api/links/:id/stats` returns total clicks, clicks per day for the last 7 days (zero-click days included) and the top 5 referrers.
- Count in the database, not by loading every click into Node.

Data and tests

- Migrations committed, with a unique index on the code.
- A seed script for a demo account with a few links and clicks spread over the past week.
- Tests for at least: one user can't access another's link, alias uniqueness, and redirect, 404 and 410 behaviour.
- A `GET /health` endpoint.

Frontend requirements

A small dashboard to create and manage links and see how each one is doing.

- Sign up, log in, log out. Logged-out users get sent to login.
- Create form: URL, optional alias, optional expiry. Show the API's errors next to the right field, e.g. "alias already taken".
- Links list: short URL with a copy button, destination, created date, click count and status (active or expired). Edit and delete, with a confirmation before deleting.
- Link details page: total clicks, a clicks-per-day chart for the last 7 days and the top referrers.

- Loading, empty and error states throughout, including what happens when the session expires.
- Works on a laptop and doesn't break at phone width. Any UI kit or chart library is fine.

Deployment

Both apps live on public HTTPS URLs, and a short link created on the live site must actually redirect for anyone who opens it.

- Frontend and API deployed separately. Free tiers are fine (e.g. Vercel or Netlify for the frontend; Render, Railway or a VPS for the API; Neon or Supabase for Postgres).
- Config through environment variables, no secrets in the repo, and a `.env.example` for both apps.
- CORS on the API only allows your frontend's origin. The redirect route stays public.
- The live database has the demo account seeded.
- If your API host sleeps when idle, mention it in the README so we don't read a cold start as a bug.
- Keep everything running for at least 7 days after you submit.

Optional extras

Only once everything above works. One or two is plenty, and none are needed to do well.

- A downloadable QR code for each link.
- Rate limiting on link creation per user, returning 429.
- A Redis cache for code lookups on the redirect path.
- Password-protected links.
- GitHub Actions running your tests on every push.
- Docker Compose for local setup.

What to submit

Reply to the email this came with, including:

- [ ] Live frontend URL and API base URL
- [ ] Repo link(s). If private, add the GitHub username we share with you
- [ ] Demo login for the seeded account
- [ ] README covering local setup, your schema, how codes are generated and collisions handled, your 301 vs 302 choice, how click counts stay accurate, known issues, what you'd do with more time, and where you used AI tools

How we'll evaluate

A working app on the live URLs matters most. Before the call we'll spend about 15 minutes on it and try the following:

- Shorten a link, open it in a private window, and check the click shows up.
- Create a link with an alias that's already taken, and one with an invalid URL.
- Open one short link 50 times in parallel and check the count says 50.
- Open an expired link and a deleted link.
- Log in as a second account and try to view, edit and delete the first account's link by ID.
- Leave the dashboard open until the session expires and see what happens.

Area	Weight	What we look for
Works end to end on the live URLs	30	Create, redirect, count and stats all work live; demo login works
Backend correctness	25	Validation, unique codes, right status codes, accurate counts under load, stats done in SQL
Security and data isolation	15	Password hashing, JWT handling, every query scoped to the user, CORS, no secrets in the repo
Frontend	15	Clear forms and errors, loading/empty/error states, sensible component structure
Code quality, tests, README	10	Readable code, tests that would catch real bugs, an honest README
Deployment	5	Env config, migrations, health check

Total is 100. Extras add a little on top but never make up for a broken core.

The review call

If the app works, we'll set up a 45-minute call to go through it together:

1. **Demo (5 min):** a quick walk through the live app.
2. **Code and trade-offs (15 min):** expect questions like what you'd change at 10 million links and a few thousand redirects a second.
3. **Live change (20 min):** on a screen share, we'll ask for a small feature, such as a max-clicks limit per link.
4. **Your questions (5 min).**

Good luck, and have fun with it.